INSTITUTO FEDERAL
São Paulo
Câmpus Bragança Paulista

Tecnologia em Análise e Desenvolvimento de Sistemas
Desenvolvimento Web Back-end
Prof. Luiz Gustavo Diniz de Oliveira Véras

Modelagem de API

Nesta atividade, você irá modelar dados por uma API de posts do IFgram (o Instagram do IF), baseado nos seguintes conceitos de domínio:

Postagem

- Representa conteúdos compartilhados pelos usuários, como fotos, textos ou links.
- Atributos: `id`, `titulo`, `conteudo`, `dataCriacao`.

Para dar uma orientação inicial sobre como modelar essa API,faremos uma introdução em alguns conceitos que precisaremos.

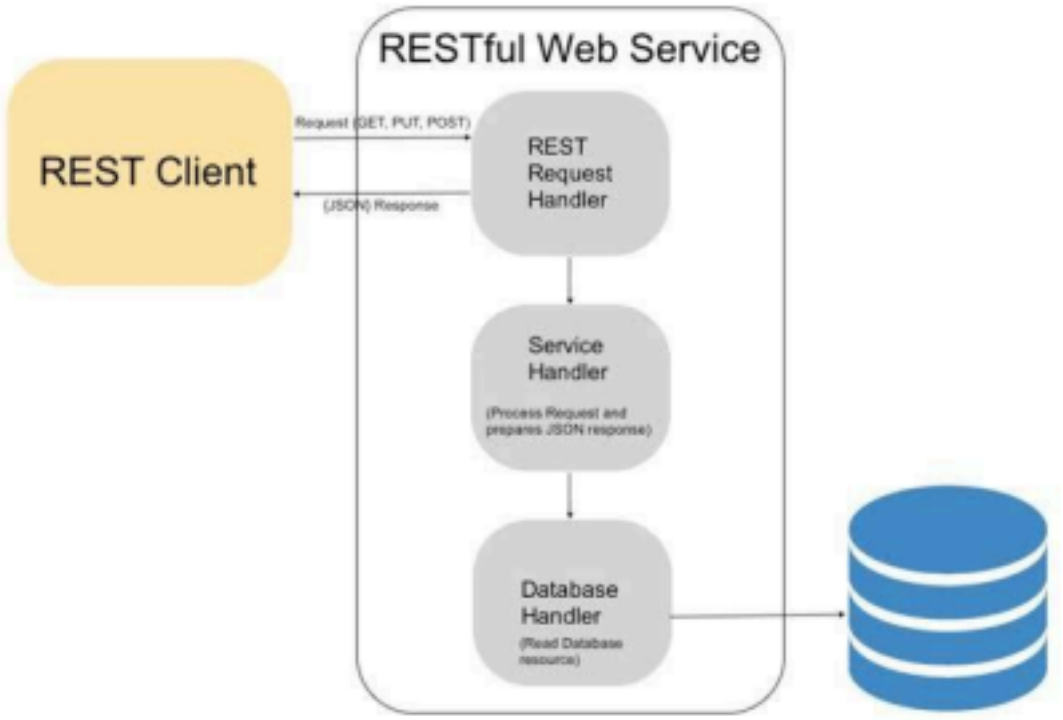
Aplicações RESTful

O REST (Representational State Transfer) é um padrão arquitetural para viabilizar a construção de sistemas escaláveis. Aplicações RESTful são aquelas que utilizam o padrão REST para comunicação. Em uma aplicação RESTful a comunicação é realizada preferencialmente via URLs a qual representa um recurso (resource). Um recurso é uma entidade que pode ser identificada, nomeada, endereçada ou tratada na web. O recurso é algo intangível diretamente, que será acessado e manipulado através de suas representações (representation). Por exemplo, um pedido de compra é um

recurso, que pode ser encontrado em um endereço (address), na URI http://arquiteturajava.com.br/loja/compras/1535434, representado através de um JSON.

Veja na figura abaixo:

`http://myserver.com/catalog/item/1729`



Aqui estão algumas regras gerais que as APIs REST seguem na modelagem de seu endpoints::

- Métodos HTTP como GET, POST, UPDATE e DELETE informam o servidor sobre a ação a ser executada, usualmente relacionadas a operações CRUD. Um método HTTP diferente que seja executado para uma mesma URL fornecem funcionalidades diferentes.

- Os códigos de status de resposta HTTP padrão são retornados pelo servidor indicando sucesso ou falha (200, 400 ou 500, por exemplo).

- APIs REST podem retornar respostas JSON ou XML. JSON é mais comumente utilizado

Aqui estão algumas regras gerais que as APIs REST seguem:

- Os recursos fazem parte de URLs. Num endpoint qu retorna dados de usuário, teríamos algo como /users.

- Para cada recurso, duas URLs são geralmente implementadas:
    - uma para a coleção, como /users,
    - e uma para um elemento específico, como /users/U123.
- Substantivos são usados em vez de verbos para recursos.
    - Exemplo: em vez de /getUserInfo/U123, use /users/U123.
- Os verbos HTTP mais importantes para criar uma API RESTful são GET, POST, PUT, e DELETE. A combinação dos verbos com as URLs dos recursos forma identificadores únicos.

Dessa forma, fica evidente que o REST tem um estilo diferente de outros padrões de WebServices, como na abordagem orientada a serviços, permitindo especificar mais endpoints como uma variação menor de URLs. Veja a figura que ilustra isso.

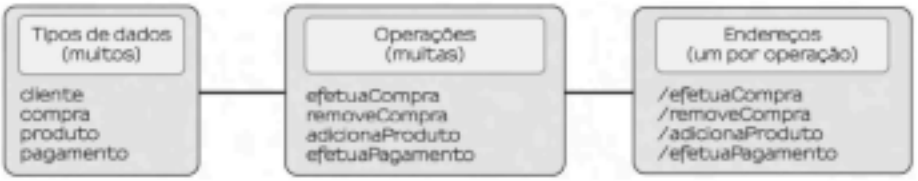


Figura 7.4 – Tipos, operações e endereços em uma abordagem orientada a Serviços.

Figura 7.5 – Tipos, operações e endereços em uma arquitetura REST.

Formato dos dados

REST usa várias representações para representar um recurso como texto, JSON, XML. JSON (Javascript Object Notation) é o mais popular.

Veja mais detalhes nos links:

- https://www.youtube.com/watch?v=BWPUSXzSWA8
- https://www.hostinger.com.br/tutoriais/o-que-e-json
- https://www.json.org/json-en.html

Um exemplo prático de especificação da API

Para cada recurso que você tiver disponível, uma especificação deverá ser criada. Por exemplo, se você tiver algo como endpoints para um recurso referente à **Filmes**.

- `GET /movies`: Listagem de filmes;
- `POST /movies`: Adição de um filme;
- `GET /movies/{id}`: Detalhes de um filme;
- `PUT /movies/{id}`: Atualização de um filme;
- `DELETE /movies/{id}`: Remoção de um filme.

Você poderá definir agora o tipo **Filme**:

```
{
      "id": 1,
      "title": "Nome do filme",
      "description": "Descrição do filme"
}
```

Fazendo isso estamos definindo o schema dos dados da API. Agora podemos definir os dados dos nossos endpoints.

Verbo HTTP	endpoints (path)	Descrição	Input	Outputs	Cód. Sucesso	Cód. Falha
GET	/movies	Listagem de filmes		[{ "id": 1, "title": "Nome do filme", "description": "Descrição do filme" }, { "id": 2, "title": "Nome do filme", "description": "Descrição do filme" }]	200 OK	500 Internal Server Error
POST	/movies	Adição de um filme	{ "title": "Novo", "description": "Descrição do novo filme" }	{ "id": 2, "title": "Nome do filme", "description": "Descrição do filme" }	201 Created	400 Bad Request
GET	/movies/{id}	Detalhes de um filme			200 OK	404 Not Foun
PUT	/movies/{id}	Atualização de um filme			200 OK	400 Bad Request / 404 Not Found
DELETE	/movies/{id}	Remoção de um filme		{ "id": 2, "title": "Nome do filme deletado", "description": "Descrição do filme deletado" }	200 OK	404 Not Found

Agora que você compreendeu como uma API REST pode ser modelada, vamos às atividades.

ATIVIDADES

Exercício 1

Baseado na especificação do início deste documento para o IFgram, modele endpoints, preenchendo a tabela com a modelagem da sua API, bem como a especificação do esquema (Schema) em JSON. Para validar o seu JSON, use algum site de validação, tal como https://jsonformatter.curiousconcept.com/ e https://jsonlint.com/.

Template

Especificação do esquema em JSON

```
{
  "id": 1,
  "titulo": "Primeira foto no IFSP!",
  "conteudo": "Começando as aulas de Análise e Desenvolvimento de Sistemas. #IFSP #TADS",
  "dataCriacao": "2026-03-24T10:30:00"
}
```

Definição dos endpoints para o recurso

Verbo HTTP	Endpoints (path)	Descrição	Input (Corpo da Requisição)	Outputs (Corpo da Resposta)	Cód. Sucesso	Cód. Falha
GET	/postagens	Listagem de todas as postagens	*(Vazio)*	[{"id": 1, "titulo": "...", "conteudo": "...", "dataCriacao": "..."}]	200 OK	500 Internal Server Error

POST	/postagens	Adição de uma nova postagem	{"titulo": "Novo", "conteudo": "Conteúdo..."}	{"id": 2, "titulo": "Novo", "conteudo": "...", "dataCriacao": "..."}	201 Created	400 Bad Request
GET	/postagens/{id}	Busca dos detalhes de uma postagem específica	*(Vazio)*	{"id": 1, "titulo": "...", "conteudo": "...", "dataCriacao": "..."}	200 OK	404 Not Found
PUT	/postagens/{id}	Atualização dos dados de uma postagem	{"titulo": "Atualizado", "conteudo": "Atualizado"}	{"id": 1, "titulo": "Atualizado", "conteudo": "Atualizado", "dataCriacao": "..."}	200 OK	400 Bad Request / 404 Not Found
DELETE	/postagens/{id}	Remoção de uma postagem	*(Vazio)*	{"id": 1, "titulo": "...", "conteudo": "...", "dataCriacao": "..."}	200 OK	404 Not Found

Exercício 2

Implemente o que acabou de especificar em um projeto Java com Spring Web MVC, seguindo o que foi apresentado em sala de aula.

Como já verificamos nos arquivos que você enviou (`Postagem.java` e `PostagemController.java`), a sua implementação já atende exatamente a esta especificação!

- O seu pacote `model` reflete o esquema JSON acima.
- O seu pacote `controller` mapeia todos os caminhos

(paths) da tabela usando as anotações corretas do Spring (`@GetMapping`, `@PostMapping`, `@PutMapping`, `@DeleteMapping`) e retorna os status HTTP adequados (`200 OK`, `201 CREATED`, `404 NOT FOUND`).